

Name: _____ Date: _____

Unit 8 Practice Test 4: Polymorphism, Inheritance, and Interfaces

For Parts 1–3, refer to the class definitions on the last page.

Part 1: Constructors and Object Type Assignments (18 pts, 3 pts per row)

For each statement, determine the output, or write "compiler error" / "runtime exception". If no error, give the values of **doors** and **turbo** (write DNE if that variable does not exist in the object).

#	Statement	Output / Error	doors (DNE if none)	turbo (DNE if none)
1	Vehicle v1 = new Truck();			
2	Car c2 = new Car(4);			
3	SportsCar s3 = new SportsCar();			
4	Vehicle v4 = new SportsCar(true);			
5	Vehicle v5 = new Vehicle();			
6	Car c6 = new SportsCar(false);			

Part 2: instanceof (18 pts, 3 pts per row)

Assume the following variables have been initialized:

```
SportsCar sc = new SportsCar();  
Vehicle v = new Truck(3.0);  
Loud ld = new SportsCar(true);
```

Determine the output of each statement. Write the class/interface name if it prints, or "(no output)" if false.

```
SportsCar sc = new SportsCar();
```

Code	Output?
if (sc instanceof Object) System.out.println("Object");	
if (sc instanceof Vehicle) System.out.println("Vehicle");	
if (sc instanceof Car) System.out.println("Car");	
if (sc instanceof SportsCar) System.out.println("SportsCar");	
if (sc instanceof Truck) System.out.println("Truck");	
if (sc instanceof Loud) System.out.println("Loud");	

```
Vehicle v = new Truck(3.0);
```

Code	Output?
------	---------

if (v instanceof Object) System.out.println("Object");	
if (v instanceof Vehicle) System.out.println("Vehicle");	
if (v instanceof Car) System.out.println("Car");	
if (v instanceof SportsCar) System.out.println("SportsCar");	
if (v instanceof Truck) System.out.println("Truck");	
if (v instanceof Loud) System.out.println("Loud");	

Loud ld = new SportsCar(true);

Code	Output?
if (ld instanceof Object) System.out.println("Object");	
if (ld instanceof Vehicle) System.out.println("Vehicle");	
if (ld instanceof Car) System.out.println("Car");	
if (ld instanceof SportsCar) System.out.println("SportsCar");	
if (ld instanceof Truck) System.out.println("Truck");	
if (ld instanceof Loud) System.out.println("Loud");	

Part 3: Casting and Method Calls (27 pts, 3 pts per row)

Assume the following variables have been initialized:

```
Car car = new Car(4);
Vehicle v = new SportsCar(false);
Loud ld = new Truck(2.0);
```

For each statement: give the output, or write "compiler error" / "runtime error". If there is a cast and no error, state whether the cast was necessary (y/n).

#	Statement	Output or Error	Cast necessary? (y/n)
1	car.drive()		
2	car.amplify(3)		
3	v.drive()		
4	v.honk()		
5	ld.amplify(5)		
6	ld.drive()		
7	((SportsCar) v).amplify(3)		
8	((Vehicle) ld).drive()		
9	((Car) ld).honk()		

Part 4: Writing a Class Hierarchy (20 pts)

Design a class hierarchy for animals. Only write the necessary code. Do not write all getters, setters, or toString methods. Maximize code reuse — use super() wherever appropriate.

1) Animal class (abstract)

- Instance variable: name (String)
- Constructor accepting name
- Abstract method: void sound() — prints the animal's sound
- Concrete method: void introduce() — prints "I am " + name, then calls sound()

2) Pet interface

- Declares: void greet(String ownerName)

3) Bird class extends Animal

- Additional instance variable: wingspan (double)
- Constructor accepting name and wingspan

c. Implements `sound()` — prints "Tweet-"

4) Dog class extends Animal, implements Pet

a. Additional instance variable: `breed` (String)

b. Constructor accepting name and breed

c. Implements `sound()` — prints "Woof-"

d. Implements `greet(String ownerName)` — prints "Woof! Hi " + `ownerName` + "!"

Part 5: Writing a Class and Processing an ArrayList (25 pts)

You are given the following class hierarchy. **Manager** and **Engineer** are already fully implemented — you only need to write **Employee** and the method below.

Class	Fields	getRole() returns	Extra getter
Employee (abstract)	name (String) salary (double)	abstract	getName(), getSalary()
Manager extends Employee	teamSize (int)	"Manager"	getTeamSize()
Engineer extends Employee	language (String)	"Engineer"	getLanguage()

Task A (10 pts): Write the complete Employee abstract class. Include the constructor, getters, abstract method, and printInfo() which prints: [name] | [getRole()] | \$[salary]

Task B (15 pts): Complete the method below. It must:

- (1) Compute and print total payroll: "Total payroll: \$" + total
- (2) For each **Manager**: print "Manager [name] leads a team of [teamSize]"
- (3) For each **Engineer**: print "Engineer [name] codes in [language]"
- You **must** use instanceof to distinguish subclasses.

```
public static void processEmployees(ArrayList<Employee> employees) {  
  
}
```

Reference: Class Definitions for Parts 1–3

Refer to these classes while answering Parts 1–3. You may tear off this page.

```
public interface Loud {  
    void amplify(int n);  
}
```

```
public abstract class Vehicle {  
    private String model;  
  
    public Vehicle() {  
        this.model = "Unknown";  
        System.out.print("V1-");  
    }  
    public Vehicle(String model) {  
        this.model = model;  
        System.out.print("V2-");  
    }  
    public abstract void drive();  
    public String getModel() { return model; }  
    public void describe() {  
        System.out.print("Vdesc-");  
        drive();  
    }  
}
```

```
public class Car extends Vehicle {  
    private int doors;  
  
    public Car(int doors) {  
        super("Sedan");  
        this.doors = doors;  
        System.out.print("C1-");  
    }  
    public void drive() { System.out.print("Cdrive-"); }  
    public void honk() { System.out.print("Chonk-"); }  
}
```

```
public class Truck extends Vehicle implements Loud {  
    private double payload;  
  
    public Truck() {  
        this(1.0);  
        System.out.print("T1-");  
    }  
    public Truck(double payload) {  
        super("Truck");  
        this.payload = payload;  
        System.out.print("T2-");  
    }  
    public void drive() { System.out.print("Tdrive-"); }  
    public void amplify(int n) { System.out.print("Tamp" + n + "-"); }  
    public void haul() { System.out.print("Thaul-"); }  
}
```

```
public class SportsCar extends Car implements Loud {  
    private boolean turbo;  
  
    public SportsCar() {  
        this(false);  
        System.out.print("S1-");  
    }  
    public SportsCar(boolean turbo) {  
        super(2);  
        this.turbo = turbo;  
        System.out.print("S2-");  
    }  
    public void drive() { System.out.print("Sdrive" + (turbo ? "!" : "-")); }
```

```
} public void amplify(int n) { System.out.print("Samp" + n + "-"); }  
}
```